Contents

Plan: morph **aicesat** into a small set of libraries around one collection framework 1
0. What the code says (and where it contradicts the earlier "Index as Data" sketch) . . . 1
1. Package decomposition . . . 2
2. The framework: one pipeline, a descriptor, pure cores and interpreters . . . 4
3. Test strategy . . . 9
4. Migration path . . . 10
5. What not to do, and what actually enforces the discipline . . . 13
6. A sixth axis: the grid is a parameter (H3, S2, rHEALPix, ...) . . . 13

Plan: morph **aicesat** into a small set of libraries around one collection framework

Drafted 2026-10-17 by a design agent against the code at `fd99853`, on Kevin's brief: separation of concerns into reusable libraries; pure functional idioms (actions as data, effects at the edges, replayable behaviour, process parallelism over values); clean code and complete tests; performance first; one unified pipeline driven by a collection descriptor with pluggable stages and configuration as a value. Companion to the approved performance batch 2 (`immutable-bubbling-naur.md`), whose items 1 and 4–6 are steps A–B and the seed of F below.

0. What the code says (and where it contradicts the earlier "Index as Data" sketch)

The leaves are already functional and native-speed: `access.coalesce`, `access.decode_chunk`/`unshuffle` (zlib + numpy transpose), `index._chunk_manifest` (h5py `chunk_iter`), `planner.cells_for_bbox`, `_cells_vectorized` (h3ronpy), `index.typed_table`, the six parsers. Everything that hurts is the spine, and the spine is copied:

Copy	Where	Count
Build driver (plan, login, search, resume, pool, claim, rollup)	`build_atl06.build_bbox:35`, `build_gedi.build_bbox:32`, `scripts/build_{glas,icessn,gpstruth}_index.py` `main()`, `scripts/build_index.py` + `index.ensure_index:282`	5 arrangements
Resume set (`pq.read_schema` on every footer, delete stale, invalidate claim)	`index.indexed_granules:246`, `index_atl06:42`, `index_glas:56`, `index_gedi:152`, `index_icessn:37`, `index_gpstruth:50`	6
Per-granule builder (open, structure, positions, cells, rows, write)	`index.build_granule_index:131`, `index_atl06:78`, `index_glas:98`, `index_gedi:172`, `index_icessn:89`, `index_gpstruth:94`	6 (3 shapes)
Ingest loop (`_index_rows` -> have -> read lake -> `_ingest_granule` -> `_keep` -> `submit_writes` -> concat)	`index_atl06.fetch_bbox:323`, `index_glas:254`, `index_gedi:365`, `index_icessn:244`, `index_gpstruth:238`, plus `planner._ensure:273` for ATL03	6
H3 assignment of the same points twice	`_keep` (`index_atl06:385`), `_parse_span_points` (`index_icessn:237`, `index_gpstruth:217`) and again in `lake._cell_tables:114`	all index missions
Pool start method	`spawn` only in `index.ensure_index:304`; the other five take the platform default (fork on the box)	5 of 6

Copy	Where	Count
Per-task pickling	`functools.partial(_index_one, res=res, cells=cells)` with `chunksize=1` (`build_atl06:92`) pickles the whole addressing-cell set once per granule	5 of 6

Import graph (AST, including function-local imports): one 13-module SCC — `coreg, coverage, gpstruth, icessn, index, index_*, lake, planner, scene`. `coverage.py` is CMR client + collection registry + manifest + claim gate + presentation (`claim_regions`, `fit_to_coverage`, `FOOTPRINTS`). `lake.py` is layout + writer + DuckDB meta + eviction + stats + read queries. 391 `monkeypatch.setattr` sites in tests, most on module-level path constants (`lake` 62, `cache` 29, `index` 29, `api` 29, `coverage` 25, `index_atl06` 24).

Where the as-data sketch needs correcting:

1. The claim cannot stop being consulted. `plan_build` comparing keys is right, but `index_covers_area` (`coverage.py:424`), `claim_regions:480`, `coverage_gap:558`, `fit_to_coverage:667`, `survey.claimed`, and the JS all read `_build.json`. The claim stays as a *derived, cached* artefact written by `claim(records)` in the same shape; the readers do not change in that step.
2. Content-addressed keys break the directory glob. `coverage.read_parquet_src:384` falls back to `read_parquet('{d}/*.parquet` and `_source_parquets:241` globs for the manifest. With accumulated versions in the store, every glob reader would see stale rows. The manifest must carry the file key and the glob fallback must be deleted in the same step.
3. Job key needs a per-collection parameter hash. GPS stamps `metadata_sha1` (`index_gpstruth:159`) because the sled geometry is in the rows. The key is `sha1(name, revision, schema, res, cellset_sha, params_sha)`; `params_sha` is a descriptor slot, empty for five collections.
4. An index file is derived but not disposable in practice. Its derivation is the whole build cost (48,838 ATL03 files on the box). GC of unreachable keys must be explicit and conservative, never on a read path.
5. "Points carry their cell" already half exists for ICESSN/GPS (`_parse_span_points` returns `cell`); the second pass is `lake._cell_tables`. The fix is in the lake's write API (accept a `cell` column), not in the planner.
6. GLAS reads positions through h5py's block cache (**`index_glas:139`**), the other HDF5 builders through the chunk map and `reader.fetch`. Unifying moves GLAS onto the measured path; it must be benchmarked, not assumed.
7. **`sample_evenly`** and whole-granule paths are already dead; `test_index_only.py` enforces it by regex. That test becomes an import-linter contract (query layer may not import the CMR module), which is stronger and cheaper.

1. Package decomposition

The owner's expectation (few libraries, collection framework central) and goal (a) pull apart only at two seams: H3 geometry and the point cache. Both are used by the app *without* the framework (`api.region_hex`, `survey`, the Lake tab), so they must sit below it or the app drags the framework in for a hex outline. Everything else that was proposed as a separate library (planner, query engine, builder, store) is a *module layer inside the framework*, separated by import-linter contracts rather than packaging. That gives the separation of concerns without five `pyproject.toml`s for things that share one row schema.

Package	Responsibility (one line)	Public API	Depends on	Who else would reuse it
`h3geo` (library, ~400 lines)	Pure H3 geometry over areas and cell sets	`cells_for_bbox`, `cells_vectorized`, `claim_res`, `coverage_cells`, `addressing_cells`, `search_polygon`, `cells_bbox`, `compact`, `cells_within`, `shrunk_ring`, `polygon_bbox`, `points_in_polygon`, `normalize_area`	h3, h3ronpy, shapely, numpy	Anyone indexing anything by H3; the app's survey/region code
`edcloud` (library, ~900 lines)	Earthdata Cloud mechanisms: EDL token, CMR search -> `SearchRecord`, STS creds, range-GET mechanisms (crt/aiobotocore/s3fs/boto3), presign, whole-object GET, h5py file opener; plus the pure parts: `coalesce`, `decode_chunk`, `unshuffle`, `chunk_manifest`, `filters`	`Credentials`, `login(cfg)`, `cmr.search(spec) -> SearchRecord`, `GranuleRef`, `Fetcher` protocol + `make_fetcher(cfg)`, `fetch(url, ranges)`, `read_all`, `open_hdf5`, `coalesce`, `decode_chunk`, `chunk_manifest`	requests, earthaccess, awscrt, s3fs, h5py, numpy	Any project reading NSIDC/LP DAAC granules by byte range (this is the h5coro-adjacent piece)
`pointlake` (library, ~900 lines)	Cell-partitioned Parquet memo of decoded points with a DuckDB key table, background writer, budget eviction, stats	`Lake(cfg)`, `write_chunk(ChunkWrite)`, `submit/drain`, `keys(mission, granules) -> set`, `query_points(...)`, `cell_stats`, `evict`, `enforce_limit`, `settings`, `ChunkWrite`	pyarrow, duckdb, numpy, `h3geo`	Any cell-keyed point cache; it never sees a collection
`cellindex` (library, the framework, ~2,500 lines)	One pipeline for sub-granule H3 indexing and index-driven ingest, driven by a `Collection` descriptor	`Collection`, `Layout` kinds, `Config`, `BuildSpec`, `Job`, `IndexFile`, `plan_build`, `build_job`, `run_build`, `IndexStore`, `claim`, `manifest`, `Query`, `QueryPlan`, `plan_query`, `execute`, `render`	`h3geo`, `edcloud`, `pointlake`, pyarrow, duckdb, h5py	Another altimetry/lidar project adding its own descriptor

aicesat (application)	Scenes, analysis, API, HTTP+stream, MCP, TUI, browser UI, deploy	api.*, server.main, tui.main	all of the above + pyproj, scipy, rasterio, mcp, rich	Not reused as a library

DAG: h3geo <- pointlake <- cellindex -> edcloud; aicesat -> all four. No cycles; edcloud and h3geo have no internal dependencies.

Analysis (coreg, gia, timeseries, lakelevel, dem, imagery, scene.frame_crs/local_frame/to_local) stays in the app as aicesat.science (pure maths) and aicesat.layers (effectful DEM/imagery readers). It is already pure at the function level; promoting it to a package has no second consumer today and would cost a release cycle per change. Revisit when one appears.

Repo layout: one repo, uv workspace.

```
pyproject.toml            # app "aicesat", [tool.uv.workspace] members = ["packages/*"]
packages/h3geo/           # each: pyproject.toml, src/<pkg>/, tests/
packages/edcloud/
packages/pointlake/
packages/cellindex/
src/aicesat/              # the app
scripts/, deploy/, docs/
```

[tool.uv.sources] cellindex = { workspace = true } etc.; uv sync installs members editable, so deploy/bootstrap.sh and uv run python scripts/serve.py on the box are unchanged. Separate repos only when an external consumer needs a release cadence different from the app's. Recommend: monorepo for at least the first year.

2. The framework: one pipeline, a descriptor, pure cores and interpreters

2.1 Performance contract (first, because it decides the shape)

Hot path	Today	After	Gate
HDF5 structure walk (superblock, groups, chunk_iter)	h5py over fsspec 1 MiB block cache, in a worker	Same code (edcloud.hdf5.chunk_manifest, open_hdf5), same process-per-granule. Descriptor adds one dict lookup per *dataset*, none per point	scripts/bench_index_fetch.py
Range GETs	RangeReader built per granule; CRT client cached per (token, region) per process (access.py:727)	Fetcher built once per worker process in the pool initializer; CRT path unchanged. The per-granule requests.Session construction disappears	bench_coalesce.py, bench_fetch_mechanisms.py
zlib + unshuffle	numpy/zlib	unchanged function, called from one decode slot	byte-identity test vs h5py (exists: probe_range_get.py)
H3 assignment	h3ronpy, run twice per fetched chunk	once; points carry cell into pointlake.write_chunk	bench_ingest_phases.py (materialize_seconds)
Parquet writes	background _Writer, per-cell tables	unchanged mechanism, lives in pointlake	bench_lake_params.py

Hot path	Today	After	Gate
DuckDB manifest / `index_files_for_cells` / index scan	unchanged	unchanged SQL; result handed to the pure planner as an Arrow table, grouped with numpy, not `to_pylist()` dict loops (`planner.py:289`)	`bench_access.py`
Build start	`pq.read_schema` on every file (33k ATL06 footers) + sidecar stats	`os.scandir` of key names	time-to-first-job on the box
Pool task payload	earthaccess object or `partial(..., cells=<set>)` per task	`Job` = 4 strings + 3 ints; cellset loaded once per worker from the store by sha (initializer)	parent RSS during a Greenland build
Start method	fork (5 of 6)	spawn everywhere, one `run_build`	`test_index_pool_memory.py` pattern

Nothing in the purity split puts Python in a per-point loop: the pure functions take and return numpy/Arrow arrays; the per-(chunk, cell) row emission loops (`index_atl06:150`, `index.py:209`) stay per-row as today (thousands, not millions) and can later move to `pyarrow.compute` group-bys without changing the contract. Spawn costs ~0.4 s per worker to import h5py/pyarrow; amortised over any real build.

Note (from the performance plan's review): the pool `initializer=` kwarg is rejected by today's test fakes; the performance batch passes only `max_workers` and `mp_context`. The initializer belongs to step G, when the fakes are replaced by an `InlinePool`.

2.2 Values

```python
# cellindex/values.py (frozen, slots, picklable by reference to module-level functions)
@dataclass(frozen=True, slots=True)
class GranuleRef:  name: str; url: str; s3url: str; revision: str; kind: str = "data"   # kind: data|metadata
@dataclass(frozen=True, slots=True)
class BuildSpec:   collection: str; res: int; schema: str; fine_cells: tuple[int, ...]; addressing: tuple[int, ..
                   ring: tuple[tuple[float, float], ...]; bbox: tuple[float, float, float, float]; window: tuple[
@dataclass(frozen=True, slots=True)
class SearchRecord: spec: BuildSpec; at: str; granules: tuple[GranuleRef, ...]        # PRIMARY, append-only
@dataclass(frozen=True, slots=True)
class Job:         collection: str; granule: GranuleRef; res: int; schema: str; cellset_sha: str; params_sha: str
                   @property key(self) -> str: sha1(...)[:20]
@dataclass(frozen=True, slots=True)
class Outcome:     key: str; rows: int; error: str | None; seconds: float; requests: int; bytes: int
@dataclass(frozen=True, slots=True)
class Query:       collection: str; bbox: ...; polygon: ...; window: ...; res: int; clip_cells: bool; force: bool
@dataclass(frozen=True)
class QueryPlan:   query: Query; want_cells: tuple[int, ...]; rows: pa.Table; have: frozenset[tuple]; todo: tuple
                   by_url: Mapping[str, tuple[int, ...]]   # url -> row indices; n_lake: int; want_only: tuple|No
```

`SearchRecord` is stored as JSON at `index/<coll>/_searches/<sha(spec,at)>.json`. The 24 h pickle of earthaccess objects (`coverage.search:94`) dies; `GranuleRef`s are made at the door of `edcloud.cmr.search` so nothing downstream ever holds an earthaccess object (closes the fork-memory hazard at its source, whatever the start method).

2.3 The Collection descriptor

```python
# cellindex/collection.py
@dataclass(frozen=True, slots=True)
class Dataset:
```

```
    key: str            # index-row / points column name
    path: str           # HDF5 path relative to the group, or CSV field index as str
    role: str = ""      # "lat" | "lon" | "h" | "t" | "quality" | ""  (what the generic stages need to find)

@dataclass(frozen=True, slots=True)
class HDF5Chunked:      # ATL03, ATL06, GLAS: every dataset of a group shares one chunk map
    groups: tuple[str, ...]          # beams; ("",) for GLAS (Data_40HZ is in the paths)
    reference: str                   # dataset key whose chunking is the group's ("latitude", "h_ph", "lat")
    positions: Callable              # (h5group, datasets, fetch) -> Positions(lat, lon, lo, hi)   # the ATL03 qu
    group_scalars: Callable | None   # (h5file) -> {"sc_orient":..., "sdp_epoch":...}

@dataclass(frozen=True, slots=True)
class HDF5Spans:        # GEDI: datasets differ in chunking; rows are spans at the union of boundaries
    groups: tuple[str, ...]; positions: Callable; quality_field: Callable   # index_gedi.quality_field

@dataclass(frozen=True, slots=True)
class TextLines:        # ICESSN, GPS: whole file scanned once, rows are line byte-spans per cell
    scan: Callable      # (bytes) -> LineScan(lat, lon, start, end, header: bytes|None, first: bytes|None)
    header_in_rows: bool

@dataclass(frozen=True, slots=True)
class Collection:
    key: str; mission: str; short_name: str; version: str
    res: int; schema: str
    window: tuple[str, str] | None
    layout: HDF5Chunked | HDF5Spans | TextLines
    datasets: tuple[Dataset, ...]
    parse_name: Callable[[str], dict]                # start/rgt/cycle or gdate...
    date_of: Callable[[str, dict], str]              # (granule name, scalars) -> "YYYYMMDD"; GLAS reads first
    row_extras: tuple[str, ...] = ()                 # extra per-row columns (ICESSN itrf_year, GPS geometry)
    decode: Callable                                 # (raws: dict, row) -> Points (lon, lat, h, t, extras) ful
    quality: Callable | None                         # (Points) -> bool mask  (ATL06 quality==0, GLAS sat<=2, G
    point_extras: tuple[str, ...] = ()               # lake extras: ("quality",) / ("sn_slope","we_slope","itrf
    job_params: Callable | None = None               # (SearchRecord, io) -> (params: Mapping[name, Any], param
    select: Callable | None = None                   # (Query, rows) -> rows  e.g. strong-only / full-power pre
```

Everything function-valued is a module-level function in `cellindex.collections.<coll>` or in `cellindex.slots` (so a `Job` carrying `collection="ATL06"` pickles as a string and the worker looks the descriptor up in the registry). Presentation (label, colour, flag, default, epoch string, `build_priority`) is a separate `aicesat.registry.Presentation` keyed by `Collection.key`: the framework never knows a colour.

ATL06, concretely:

```
ATL06 = Collection(
    key="ATL06", mission="ATL06", short_name="ATL06", version="007", res=5, schema="atl06-3",
    window=("2018-10-01", "2027-01-01"),
    layout=HDF5Chunked(groups=("gt1l","gt1r","gt2l","gt2r","gt3l","gt3r"), reference="latitude",
                       positions=slots.positions_identity,          # lat/lon per element, lo=hi=arange
                       group_scalars=slots.icesat2_scalars),        # sc_orient, atlas_sdp_gps_epoch -> strong fl
    datasets=(Dataset("latitude","land_ice_segments/latitude","lat"), Dataset("longitude","land_ice_segments/long
              Dataset("h_li","land_ice_segments/h_li","h"), Dataset("delta_time","land_ice_segments/delta_time","
              Dataset("atl06_quality_summary","land_ice_segments/atl06_quality_summary","quality")),
    parse_name=slots.parse_icesat2_name, date_of=slots.date_from_name,
    decode=slots.decode_shared_chunk,                  # generic: decode each dataset, slice [:seg_n]
    quality=slots.quality_equals_zero,
    point_extras=("quality",),
    select=slots.select_strong_only,                   # applies when Query.beams == "strong"
)
```

ICESSN (a CSV collection), concretely:

```
ICESSN = Collection(
    key="ICESSN", mission="ICESSN", short_name="ILATM2", version="2", res=5, schema="icessn-4", window=None,
    layout=TextLines(scan=icessn.scan_lines, header_in_rows=False),  # today's build_icessn_index loop over _pars
    datasets=(),                                          # no addressed datasets: a row is one (cell) byte span
    parse_name=icessn.parse_name, date_of=slots.date_from_name,
    row_extras=("itrf_year",),                            # icessn.itrf_year_from_bytes(header) at build
    decode=icessn.parse_span_points,                      # bytes -> platelets (track==0, finite, RMS<50), t from g
    quality=None, point_extras=("sn_slope","we_slope","itrf_year"),
)
```

GPS differs from ICESSN only in `header_in_rows=True`, `row_extras=("header","rinex_key","arp_to_sled_m","track_depth_cm"`, `job_params=gpstruth.resolve_geometry` (reads the metadata granule once in the parent; its sha becomes `params_sha`), and `decode=gpstruth.parse_span_points` (which already wraps the pyarrow `parse_rows`).

Which existing functions become slots:

Slot	Existing code
`HDF5Chunked.positions` for ATL03	`index.build_granule_index:173–199` (segments `ph_index_beg/segment_ph_cnt` -> photon ranges)
`positions_identity` (ATL06, GLAS)	`index_atl06:129–138`, `index_glas:139–151`
`HDF5Spans` span cutting	`index_gedi.chunk_spans:101`, `chunk_of:118`, `span_offset:126` (already pure, keep verbatim)
`TextLines.scan`	`index_icessn._parse_fields:63` + loop 89–118; `index_gpstruth._latlon:78`, `_gdate:86` + loop 113–124
`decode`	`index_atl06._decode_chunk:261`, `index_glas._decode_chunk:236`, `index_gedi._decode_span:312`, `planner._decode_photons:55`, `index_icessn._parse_span_points:217`, `index_gpstruth._parse_span_points:200`
time functions (inside `decode`)	`index_atl06._atlas_epoch_years:188`, `planner.GPS_EPOCH_MS` arithmetic, GLAS `J2000`, ICESSN gdate+seconds
`quality`	`quality_zero` masks in `fetch_bboxs`, `index_gedi.quality_field:93`, GLAS `MAX_SAT_FLAG`
`group_scalars`	`index.strong_beams:127` + `sc_orient/sdp_epoch` reads
`job_params`	`scripts/build_gpstruth_index.py:resolve_geometry:36` + `gpstruth.traverse_metadata_from_bytes:106`
fill handling	`index_gedi._fill/_nan_fill:137–149`, `index_glas:82–90` -> `slots.nan_fill`, driven by `Dataset` fill metadata in the row
generic row emission	the `for k, cell in sorted(set(zip(...)))` loops, written once in `build.rows_from_positions`

Adding a collection = one descriptor + at most `positions`/`scan` and `decode`. No driver, no ingest loop, no resume set, no `_index_dir`.

2.4 The two pipelines as stages

Build (`cellindex.build`, pure) and `cellindex.run` (interpreter):

```
plan(area, coll, cfg)                    -> BuildSpec            pure (h3geo)
search(spec, io)                         -> SearchRecord         EFFECT (edcloud.cmr); stored
jobs(spec, record, coll, present_keys) -> tuple[Job, ...]      pure: key not in present
build_job(job, coll, cellset, params, io) -> IndexFile(table, key, meta)  pure given io = (open_hdf5|read_all, fetch)
```

```
  layout dispatch: HDF5Chunked -> structure_shared(...) ; HDF5Spans -> structure_spans(...) ; TextLines -
> structure_lines(...)
   then rows_from_positions(positions, chunkmaps, cells_keep, res)  (one implementation)
run_build(jobs, coll, cfg)        -> tuple[Outcome, ...]  EFFECT: spawn pool, initializer=(cfg, coll.key, cellset_sh
                                                    monotonic deadline per future, 2 attempts, every exception retryable
                                                                (the ensure_index:282 policy, the only one kept)
store.put(IndexFile)              -> key                EFFECT: tmp + os.replace, write-once, name == key
claim(records)                  -> CellSet            pure fold; its cache is _build.json in today's shape
manifest(store, coll)             -> Table              fold via DuckDB COPY (coverage._build_manifest:270), cached by s
```

Query (`cellindex.query`, pure core; `cellindex.execute`, interpreter):

```
scan(query, coll, store, manifest)    -> pa.Table rows      EFFECT (DuckDB; today's _index_rows SQL, one implementatio
plan_query(query, coll, rows, have)   -> QueryPlan         pure: todo/have split, by_url grouping, want_only, select()
execute(plan, coll, cfg, lake, fetcher, on_granule=None) -> Points  EFFECT: read lake first, then map_as_completed ov
      fetch(url, ranges) -> decode(raws,row) -> assign_cells(points, res) -> keep(points, plan) -
> lake.submit(ChunkWrite with cell)
keep(points, plan, coll)                -> mask                 pure; the ONE _keep
render(plan)                          -> dict/HTML          pure; what the TUI chunk_map and the pages read
```

keep is written once; the five copies (`index_atl06:377`, `index_glas`, `index_gedi`, `index_icessn:317`, `index_gpstruth:273`) go. ATL03's `_materialize` (coreg propagation to the common epoch, `planner.py:80`) becomes an ATL03-only `transform` slot so output stays byte-identical; retiring `coreg_lon/lat` is a separate item.

2.5 Config as a value

```python
@dataclass(frozen=True, slots=True)
class Config:
    data_dir: Path
    in_region: bool; s3_mechanism: str; coalesce_gap: int; crt_gbps: int; s3_pool: int | None
    fetch_workers: int; fetch_threads: int
    index_workers: int; job_timeout_s: int; start_method: str = "spawn"
    lake_writers: int; lake_async: bool; precache_adjacent: bool; lake_max_bytes: int
    token: str | None; edl_file: Path
    per_collection: Mapping[str, CollectionOverrides]      # e.g. GEDI job_timeout_s=600, res
    now: Callable[[], datetime] = utcnow                    # injected clock

Config.from_sources(env=os.environ, cli=args, file=aicesat.env) -> Config   # merged once; precedence cli > env >
```

Doors: `scripts/* main()`, `server.main`, `tui.main`, the pool initializer, and one pytest fixture (`cfg = Config.testing(tmp_path)`). Env var names map 1:1 to today's (`AICESAT_S3_FETCH`, `AICESAT_COALESCE_GAP`, `AICESAT_FETCH_WORKERS`, `AICESAT_S3_POOL`, `AICESAT_CRT_GBPS`, `AICESAT_LAKE_WRITERS`, `AICESAT_LAKE_ASYNC_WRITE`, `AICESAT_PRECACHE_ADJACENT_CELLS`, `AICESAT_DATA_DIR`, `AICESAT_EDL_FILE`, `EARTHDATA_TOKEN`, `AWS_REGION`/`AICESAT_S3_DIRECT`), so `deploy/aicesat.env.example` is unchanged. App-only settings (`AICESAT_PORT`, `_PUBLIC_URL`, `_ACCESS_CODE`, `_PROFILE`, `_IMAGERY`, `S2_*`) go in `aicesat.AppConfig`, which embeds a `cellindex.Config`.

Customisation without subclassing or monkeypatching: `Config` holds *names* (`s3_mechanism="crt"`) resolved by the interpreter through a registry (`edcloud.mechanisms`, today's `S3_FETCH_MECHANISMS` dict) because names pickle across spawn and functions do not reliably. In-process callers (tests, benches) inject functions directly as keyword arguments: `execute(plan, coll, cfg, fetcher=FakeFetcher(blobs))`, `run_build(jobs, coll, cfg, pool=InlinePool())`, `build_job(job, coll, cellset, io=FakeIO(h5=..., bytes=...))`, `Lake(cfg, layout=my_layout)`, `plan_query(..., assign=my_cells_fn)`. Replacing the cache layout = passing a different `Layout` value to `Lake`; replacing coalescing = `coalesce` is an argument of `make_fetcher`.

Transitional rule for migration only: `cellindex.config.default()` returns the process-wide config set once at the door. Old modules read it; new code takes `cfg` explicitly. It is deleted in step G and the AST test forbids it in `packages/`.

2.6 Replay

- Build: `SearchRecord` + current `Collection` + `store.keys()` recomputes the job list exactly; `render(record)` prints what the buildmap page reconstructs by hand. A failed build is re-run by re-reading the record, not re-searching CMR (staleness is a policy on `record.at`).
- Query: `QueryPlan` serialises to a directory (`plan.json` + `rows.arrow`); `execute(load_plan(p), coll, cfg)` replays it. `bench_ingest_phases.py` becomes "record once, execute N times with `replace(cfg, lake_writers=k)`". The TUI `chunk_map/plan` ops read `render(plan)`.
- Bytes: `RecordingFetcher` wraps any `Fetcher` and writes `(url, offset, size) -> bytes` into `tests/fixtures/blobs/<coll>/`. One small granule per collection, recorded once in-region, makes decode and planner tests run offline against real bytes rather than the synthetic `_gz(...)` blobs of `test_lake_cache.py`.

2.7 Parallelism idioms

- `run_build`: `ProcessPoolExecutor(mp_context=spawn, initializer=worker_init, initargs=(cfg, coll_key, cellset_sha, params_sha))`; the initializer logs in, builds the one `Fetcher` (CRT client) for the process and loads the cellset from the store by sha. Tasks are `Job` values; results are `Outcome` values. No module singleton crosses a process boundary (today `_CRT_CACHE`, `_S3_CREDS`, `auth._auth` are per-process accidents of fork).
- `execute`: `map_as_completed` over `by_url` with threads (network-bound) exactly as today; ATL03 keeps its process pool for decode+propagate (CPU-bound) as a `group_parallel` option of `execute`, not a different code path.
- `pointlake._Writer` stays a thread pool with backpressure; it is the interpreter for `ChunkWrite` values. Making writes "return actions" would lose the backpressure that bounds memory (`WRITE_QUEUE_MAX`).

3. Test strategy

Package	Pure core tests	Interpreter tests	Property tests (hypothesis, cheap)	Target
`h3geo`	literals: `test_geom.py`, `test_planner_cover.py`, `test_sample_geometry.py`, claim-compaction tests from `test_index_only.py:_claim`	none	`compact/cells_within` preserve membership; `cells_for_bbox` ⊆ centre-contained cells; `addressing_cells` round-trips parent/child	100% lines
`edcloud`	`coalesce`, `decode_chunk`, `chunk_spans` with literals and recorded blobs; `dedup_granules/granule_name` on dict fixtures	mechanisms behind a fake S3 (moto or a `Fetcher` fake); `cmr.search` with a fake earthaccess returning fixtures; EDL token parsing (`test_auth_noninteractive.py`)	`coalesce` output covers every input range, is sorted, respects `max_gap/max_span`; decode∘encode identity on random arrays with shuffle+gzip	95% pure / 80% mechanisms (network-marked tests excluded)
`pointlake`	`_cell_tables` with a `cell` column, `concat_arrays` dtypes (`test_lake_stream_dtypes.py`)	writer ordering (`test_lake_writer.py`), eviction/limits (`test_lake_cache.py` eviction half), stats persistence, query scoping; all under `Lake(Config.testing(tmp_path))`	write then `query_points` returns the same multiset for random cells	90%

Package	Pure core tests	Interpreter tests	Property tests (hypothesis, cheap)	Target
`cellindex`	`jobs()` on literal records/keys; `rows_from_positions` on arrays; `plan_query` on Arrow literals (`todo n have = ∅`, every want cell is in lake or todo, `by_url` partitions `todo`); `keep`; `claim` fold; per-collection `decode` on recorded blobs	`build_job` with `FakeIO` (synthetic h5 via h5py in-memory `driver="core"` + recorded blobs); `run_build` with `InlinePool`; `execute` with `FakeFetcher` (the `test_lake_cache.py` matrix: cold/warm/evicted/force/polygon, byte-identical to `_fetch_direct`, for all six collections)	Job key stable under field order; `plan_query` idempotent	95% pure / 85% interpreters
`aicesat`	science modules (`test_coreg.py`, `test_timeseries.py`, `test_lakelevel.py`, `test_gia.py`, `test_dem*.py`) as they are	API/server/stream/TUI none new tests as they are, with `AppConfig.testing`; golden suite unchanged at every step		85%

Enforcement: `pytest --cov=<pkg> --cov-fail-under=<n>` per workspace member in `uv run --package <pkg> pytest`; a root `make check` runs all five plus `node tests/*.js`. The golden suite (`tests/test_golden.py`, 9 numbers against the APFS clone) is the regression gate for every migration step.

Existing 10,500 lines map as follows: ~1,900 lines (index/lake/planner/access unit tests) move to the packages with path monkeypatches replaced by `Config.testing(tmp_path)` (mechanical; 391 sites, most are `setattr(mod, "<DIR>", tmp_path/...)`); ~1,200 lines retire because what they test disappears (`test_index_enlarge.py`'s `granule_proves` parts, `test_index_status_manifest.py`'s footer-scan parts, `test_candidates_memo.py` if the memo moves to `AppConfig`, three of the per-collection ingest copies in `test_index_missions.py`, `test_index_pool_memory.py` replaced by an `Outcome`-shape test); the rest (scene/api/server/stream/TUI/JS, ~7,000 lines) stay in the app largely untouched. `test_index_only.py`'s two structural tests become import-linter contracts and are deleted; its coverage-gate tests stay.

4. Migration path

Each step lands on `main`, passes the full suite plus golden, and deploys to the box with `git pull && uv sync && systemctl restart aicesat-web`. Steps A and B are the performance batch.

Step	What	Retires	Effort	Risk
A. Config at the doors	`cellindex/config.py` (lives in `src/aicesat/config.py` for now); `Config.from_sources`; thread `cfg` through `access.RangeReader.__init__`, `lake._writer_threads`, `index_atl06.precache_adjacent`, `coverage.search`; doors in 8 scripts, `server.main`, `tui.main`; pytest fixture; `default()` transitional accessor	15 `os.environ` reads below the doors (`access.py:65,88,196–205,308,336`, `lake.py:416,424`, `index_atl06:222`, `cache.py:12`, `auth.py:20,78–89`)	2 d	Low. `cache.DATA_DIR` at import time (`tui/__init__.py:48` sets env before import) is the one trap: `Paths` must be computed from `cfg` at call time
B. GranuleRef/Job + one **`run_build`**	`values.py`; `edcloud.cmr.search` returns `SearchRecord` of `GranuleRefs` (earthaccess objects die at the door); `run_build` with spawn + initializer + cellset-by-sha; the six builders take `(job, cellset, params)`; `cells_filter(cells, res)` in all six	`build_gedi.py`, the three inline script drivers, `index.ensure_index`'s pool, per-task pickling of `cells`, five fork pools	3 d	Medium. Spawn on Linux with awscrt is proven by the ATL03 pool; verify one small build per collection on the box
C. SearchRecord stored, claim derived	`_searches/*.json`; `claim(records)` fold writing `_build.json` in today's shape (so `index_covers_area`, `claim_regions`, `survey` are untouched); empty search = record with zero granules	`write_build_manifest` mutation (`index.py:355`), `invalidate_claim`, `empty_search_covering`'s special list, the CMR pickle	2 d	Low–medium; `test_coverage_manifest.py` and `test_index_only.py` claim tests pin behaviour

Step	What	Retires	Effort	Risk
D. Content-addressed index store	file name = `Job.key`; `IndexStore.keys()`; manifest rows carry `key`; `index_files_for_cells` returns keys; delete the `*.parquet` glob fallbacks (`coverage.py:242,389`); one-shot on-box migration script (rename by footer metadata, like `scripts/migrate_index_footers.py`); explicit `gc` command (keys unreachable from any record with the current schema)	`indexed_*` ×6, `granule_proves`, `_cellsets/` sidecars (sha is in the key), schema-version deletion, `_PROVES_CACHE`, `_CELLSET_CACHE`	4 d	Highest. 120k files on the box to rename atomically; every reader must go through the manifest; GC must be conservative. Rehearse on the `data-golden` clone first
E. Collection descriptor + one builder	`collection.py`, `collections/{atl03,atl06,gla...}.py`; `build_job` with three layout structures and one `rows_from_positions`; GLAS onto chunk-map reads behind a benchmark; `coverage.collections()/index_module/_index_for` replaced by the registry	six builder bodies, `_index_dir` ×5, `coverage._index_for`, per-collection `*_INDEX_VERSION` constants, `build_script`	3 d	Medium. Row schemas must stay identical per collection (golden + `test_index_atl06_extract.py`, `test_index_gedi.py`, `test_gpstruth.py`, `test_icessn_itrf.py` pin them)
F. QueryPlan + one **execute**; lake as memo	`scan/plan_query/keep/execute/_bbox`; `point-lake.write_chunk` accepts a `cell` column; `extract` wrappers (`atl06.extract` etc.) become `execute(plan_query(...))` + cache; ATL03 `_ensure` folded in with `transform` slot	five `_fetch_direct`, `_bbox` + `plan_bbox` copies, five `_keep`, `_index_rows` ×5, `planner._ensure`, the second H3 pass, `lake.PRODUCTS`	5 d	High. Byte-identity against `_fetch_direct` for all six (the FakeReader matrix is the gate); streaming callbacks (`on_granule`/`on_plan`) must keep their contract for `api.build_scene:407-480`
G. Extract packages	uv workspace; move modules into `packages/*`; import-linter contracts; delete `default()`; split tests	module-level singletons as cross-package state	3 d	Low if A–F held the lines; mostly import churn

Step	What	Retires	Effort	Risk
H. App split	`api.py` -> scenes/jobs, analysis, admin; `Presentation` registry; `stream.py` stops reading `api._jobs`; TUI ops over the public API	`api.LEGS`, `scene.COLORS`, `time-series.MISSION_LABEL`, `survey.MISSIONS` duplicates	4 d	Medium (UI bundle rebuild; `test_build_scene.py` 500 lines)

Total: ~26 days of focused work; D and F are where the time estimates are least trustworthy. Steps A–C are a few hundred lines each and remove the fork hazard, the five drivers, the per-task pickling and the env reads on their own.

5. What not to do, and what actually enforces the discipline

Do not:

- Build an effect system. Values are for the four places replay or parallelism pays (`SearchRecord`, `Job`, `QueryPlan`, `ChunkWrite`); everywhere else, a function that takes `cfg` and `io` arguments is the whole pattern.
- Make `Collection` a class hierarchy or a plugin entry-point system. A frozen dataclass with six function-valued fields, in one module per collection, is enough and pickles across spawn.
- Hide DuckDB/Parquet/h3ronpy behind "storage interfaces". They *are* the performance; the pluggable points are the fetch mechanism, the layout value, the cell-assignment function and the coalescing policy, each a plain argument.
- Split repos, publish to an index, or version packages independently before a second consumer exists.
- Make the lake writer "pure" by deferring writes to the caller; the queue's backpressure is what bounds memory.
- Refactor the leaf functions (`decode_chunk`, `coalesce`, `chunk_spans`, the parsers). They are the parts that are already right.
- Trust purity by convention. Python enforces none of it.

Enforcement that works in Python:

1. import-linter (`[tool.importlinter]` at the root), contracts: `layers` inside `cellindex` (`values` < `collection` < `build`,`query` (pure) < `run`,`execute`,`store`); `forbidden`: `cellindex.query`, `cellindex.build`, `h3geo`, `point-lake.layout` may not import `edcloud.cmr`, `edcloud.mechanisms`, `requests`, `earthaccess`, `h5py`, `duckdb`; the app may not import `cellindex.run` from request handlers. Runs in `make check`; there is no CI today, so the Makefile target is the CI.
2. An AST purity test (the `test_index_only.py:63-103` pattern, already parsed not grepped) over the pure module list: fails on `os.environ`, `os.getenv`, `open(`, `Path.write_*`/`read_*`, `os.replace`, `time.time`, `time.monotonic`, `datetime.now`, `uuid4`, `random`, `functools.lru_cache`, `threading`, and any module-level mutable (`dict()`, `{}`, `[]`, `Lock()`) assignment.
3. mypy **`--strict`** on `values.py`, `collection.py`, `config.py`, `h3geo` from day one (small, typed, no third-party stubs needed); the rest at `check_untyped_defs` and ratcheted.
4. ruff: `select = ["E","F","I","B","UP","PL","RUF","DTZ","T20"]` (`DTZ` catches naive `datetime.now`, `T20` catches `print` in libraries, `B008` catches mutable defaults); `[tool.ruff.lint.per-file-ignores]` for scripts.
5. **`frozen=True, slots=True`** on every value; `__post_init__` validation instead of defensive code downstream.
6. Coverage floors per package via `--cov-fail-under`, and the golden suite as the behavioural floor for the app.

6. A sixth axis: the grid is a parameter (H3, S2, rHEALPix, ...)

Added 2026-10-17 at Kevin's request. Today H3 is not a dependency, it is an assumption: the cell id is an H3 `uint64` in the index rows (`h3_cell`), the lake partitions (`h3_cell=<int>/`), the coverage table and the manifest; the claim is `h3.compact_cells` and membership walks `h3.cell_to_parent`; the planner polyfills with `h3shape_to_cells_experimental`; the per-point kernel is `h3ronpy.coordinates_to_cells`; `claim_res` reasons with `average_hexagon_area`; the browser draws `cell_to_boundary` hexagons; and the names say so everywhere (`H3_RES`, `h3_cell`, `h3geo`). Making the grid a parameter is one more descriptor, and the performance contract decides what it may and may not abstract.

6.1 What a grid must provide

```python
@dataclass(frozen=True, slots=True)
class Grid:                          # dggs.Grid: one value per implementation, chosen once per deployment
    name: str                        # "h3" | "s2" | "rhealpix" | ...
    id_dtype: str                    # "u8" for H3 and S2 (64-bit ids); "str" for rHEALPix ("N2345") -> see 6.3
    resolutions: range               # the levels this grid offers (H3 0-15, S2 0-30, rHEALPix 0-..)
    cell_area_km2: Callable[[int], float]                    # mean cell area at a level (claim_res, cell-size
    cells_vectorized: Callable[[np.ndarray, np.ndarray, int], np.ndarray]   # lat, lon -> ids; THE hot kernel
    cells_for_area: Callable[[Ring, int], Sequence]          # polyfill with overlap semantics (planner.cells_f
    parent: Callable[[Cell, int], Cell]; children: Callable[[Cell, int], Sequence[Cell]]
    compact: Callable[[Sequence[Cell]], Sequence[Cell]]      # the claim's representation
    boundary: Callable[[Cell], Sequence[tuple[float, float]]]  # (lat, lon) ring, for pages, the UI and cells_bbo
    neighbors: Callable[[Cell, int], Sequence[Cell]]         # grid_disk(k): precache-adjacent, hex-reset
    to_text: Callable[[Cell], str]; from_text: Callable[[str], Cell]
```

Everything in `h3geo` becomes `dggs` with `dggs.h3 = Grid(...)` as the reference implementation; `s2` and `rhealpix` are further modules with the same shape. The framework imports `Grid`, never `h3`. The grid is part of the store's identity: `index/<coll>/_grid.json` and the lake root record `{name, version}`, and a `Job.key`/`QueryPlan` carries the grid name so a store built under one grid is never read under another. One grid per deployment; mixing grids in one lake is out of scope.

6.2 What may be abstracted and what may not (the performance contract)

Operation	Per what	Abstract?	Why
point -> cell	per point (59 M in one 8-cell ingest)	only through a vectorised native kernel	h3ronpy is Rust at ~2 M points/s/core; a pure-Python S2 or rHEALPix binding is 100-1000× slower and would make the grid unusable, not merely slower. A `Grid` without a vectorised `cells_vectorized` is rejected at import (the descriptor validates it on an array of 10^5 points under a time budget)
cells for an area, compact, parent, children, boundary, neighbors	per cell (hundreds to thousands)	yes, plain Python is fine	these run once per query or build, not per point
cell id storage and comparison	per row and per file	fixed to a 64-bit integer column	DuckDB pushdown (`cell IN (…)`), Parquet statistics and hive partition names all assume a sortable scalar; strings are 5-10× slower to filter and partition. Grids whose native id is a string are packed (6.3)

Operation	Per what	Abstract?	Why
convex cells	per (chunk, cell) run (item 6 of the performance plan)	an attribute of the grid: `Grid.convex_cells: bool`	the "one run per cell per chunk" shortcut holds for H3 hexagons, S2 cells (spherical quadrilaterals) and rHEALPix cells (quadrilaterals); a grid with non-convex cells disables the exact-run path and keeps the per-point fallback

6.3 Ids, names and partitions

- Column and partition names drop the grid: `h3_cell` -> `cell`, `h3_cell=<int>/` -> `cell=<int>/`, `H3_RES` -> `Collection.res` (a level in `Grid.resolutions`). This is a one-time rename across index schemas, lake layout, the coverage table, the manifest and the UI, which is why it rides with the schema bump and lake wipe already planned (performance batch items 5-6) rather than being a step of its own.
- Every grid gets a 64-bit integer id in storage. H3 and S2 have one natively. rHEALPix's `"N2345"` packs into 64 bits as (face: 3 bits, level: 5 bits, digits: 4 bits each up to level 14) with `Grid.to_text/from_text` as the human form; the pages and the API show the text form. The packing is the grid module's business; the framework sees integers.
- The claim is `Grid.compact(cells)` at the claim level and membership is an ancestor walk through `Grid.parent`, exactly as today, so `cells_within`, `unclaimed_cells` and the enlarging-build logic are unchanged in shape. `claim_res` uses `cell_area_km2` to pick the finest level under the cell cap, which is where H3's res-9 "~200 m" and S2's level 16 "~150 m" become comparable numbers rather than magic constants.
- The per-collection addressing level is chosen per grid by the same rule as today (a cell about the size of a chunk's footprint): H3 5 ? S2 9 ? rHEALPix 4 for ATL06's 200 km chunks. The descriptor stores it per grid: `Collection.res: Mapping[grid_name, int]`.

6.4 Where the grid enters the migration

Step E (the collection descriptor) is where `Grid` lands: `plan`, `rows_from_positions`, `keep` and `assign_cells` take `grid` from the config, and `dggs.h3` is the only implementation. Adding S2 afterwards is a new module plus a vectorised kernel (the `s2geometry` C++ bindings or a small Rust extension over the `s2` crate; `s2sphere` is pure Python and fails the import-time budget). rHEALPix has no vectorised kernel in the ecosystem today; it is the case that would justify a PyO3 extension, which the restructuring makes a 200-line package instead of a change to the framework.

Tests: the `Grid` contract gets one conformance suite run against every implementation (ids round-trip through text; `parent(children(c))` is `c`; `compact` then membership preserves every cell; `cells_for_area` covers the centre-contained cells of a box; `cells_vectorized` agrees with the scalar API on 10^4 random points and meets the time budget on 10^5). The framework's own tests run under `dggs.h3` only; a second grid runs the conformance suite and the six-collection golden ingest once, since the framework is grid-agnostic by construction.

What not to do: no grid-neutral geometry library, no `GridCell` objects per point, no runtime grid detection from ids. A grid is a value chosen once; everything per point stays a NumPy array of 64-bit integers.

Critical files

- `src/aicesat/index_atl06.py` — the reference builder (`build_atl06_index:78`) and ingest loop (`plan_bbox:270`, `fetch_bbox:323`) that become the generic `build_job`/`plan_query`/`execute`
- `src/aicesat/build_atl06.py` — the reference driver (`build_bbox:35`) that becomes `run_build`
- `src/aicesat/index.py` — `ensure_index:282` (the spawn/deadline/retry policy to keep), the claim code (`write_build_manifest:355`, `unclaimed_cells:490`, `granule_proves:620`) to retire, and the shared HDF5 helpers that move to `edcloud`
- `src/aicesat/access.py` — `RangeReader` and the mechanism registry (`S3_FETCH_MECHANISMS:825`) that become `edcloud.Fetcher`, and the env reads that move into `Config`

- `src/aicesat/lake.py` — `_Writer:444`, `_cell_tables:103`, the DuckDB meta table (`_open_meta:328`) that become `pointlake`
- `src/aicesat/coverage.py` — `search:79`, `collections:147`, `_index_for:197`, the manifest (`_build_manifest:270`, `index_files_for_cells:392`) and the glob fallbacks that must die with content-addressed keys